

Technical Note: The Four Core Laws of FractalShield — A Framework for Oracle-Free Verification and Geometric Cost Escalation V-1.2

Definition 3.1 — Technical Note

Miguel Angel Franco León

Fracta-Axis Project

<https://github.com/Fracta-Axis/Fractalylx>

Technical Note v1.2 May 2026

Companion document to FractalShield Preprint v1.0 [1]. Not peer-reviewed.

Abstract

This note provides a self-contained formal specification of **Definition 3.1** from the FractalShield framework [1]: the four minimal properties (**C1–C4**) that any function must satisfy to serve as a valid cryptographic core. For each property we state the formal requirement, its cryptographic role, the security theorem it protects, and the consequence of its violation. We include a complete security inheritance table, a minimal implementation contract, and a conformance verification checklist. The goal of this note is to lower the barrier for the community to propose and validate new FractalShield core instantiations independently of the MFSU-Crypt construction presented in the main preprint.

Contents

1 Preliminaries	4
2 The Four Laws	4
2.1 Full Statement	4
2.2 Law-by-Law Analysis	5
3 Security Inheritance	6
4 Violation Consequences	6
5 Minimal Implementation Contract	6
6 Conformance Verification Checklist	7
7 Reference Values (MFSU-Crypt)	8
8 Open Problems Relevant to Core Design	8
9 Conclusion	10
Appendix A: Reference Test Vectors	11
Appendix B: Resolution of Open Problem 7	13
Appendix c: Resolution of Open Problem 6	17
Appendix D: Resolution of Open Problem 1	20

=====

Introduction

The FractalShield framework [1] is a *core-agnostic* construction for oracle-free layered encryption. Its central architectural idea is a strict separation between (i) the *framework*—layering, shuffling, HMAC wrapper, and file format— and (ii) the *core function* F , which is the only component that varies across instantiations.

This separation has the following consequence.

Inheritance Principle. Any function F satisfying C1–C4 (Definition 2.1) inherits all four security theorems of FractalShield without modification: Integrity (Thm. 5.1), IV Uniqueness (Thm. 5.2), Oracle-Free Verification (Thm. 5.5), and IND-CCA2 (Thm. 5.6).

Purpose of this note. The main preprint proves the security theorems in terms of C1–C4 abstractly. This document explains *why* each property is necessary, what breaks when it is absent, and how to verify conformance for a candidate core. Isolating Definition 3.1 here with its own identifier allows future works to cite the abstract contract without importing the MFSU-specific material.

Existing instantiations. The preprint [1] provides two instantiations:

- **MFSU-Crypt:** core derived from a fractional stochastic PDE; 8 MB scratchpad; ≈ 1.9 attempts/sec (Python).
- **Argon2id-Shield** (Appendix D of [1]): Argon2id [3] as F ; 64 MB scratchpad; 10+ years of public cryptanalysis; demonstrates core-agnosticism with a well-audited primitive.

Candidate future cores. Gray–Scott reaction-diffusion systems, generalised Lorenz flows, script [6], and novel constructions. Any function meeting C1–C4 qualifies.

1 Preliminaries

We follow the notation of [1]. κ is the security parameter; $\text{negl}(\kappa)$ denotes a negligible function; PPT stands for probabilistic polynomial time; $x \xleftarrow{\$} S$ denotes uniform sampling from S ; U_n is the uniform distribution over $\{0, 1\}^n$.

Definition 1.1 (Memory-Hardness [2]). A function f is (t, m) -memory-hard if every algorithm computing f in time t requires at least m bits of memory. A device with G bytes of RAM can run at most $\lfloor G/m \rfloor$ concurrent evaluations.

Definition 1.2 (Pseudorandom Generator). $G : \{0, 1\}^\kappa \rightarrow \{0, 1\}^{\ell(\kappa)}$ is a PRG if for all PPT distinguishers D :

$$|\Pr[D(G(k)) = 1] - \Pr[D(U_\ell) = 1]| \leq \text{negl}(\kappa).$$

Definition 1.3 (SUF-CMA MAC). A MAC is *strongly unforgeable under chosen-message attack* (SUF-CMA) if no PPT adversary produces a valid tag on any new message with non-negligible probability.

2 The Four Laws

2.1 Full Statement

Definition 2.1 (Valid FractalShield Core Function). A function

$$F : \{0, 1\}^\kappa \times \{0, 1\}^{128} \times \mathbb{N} \longrightarrow \{0, 1\}^*$$

is a *valid FractalShield core* if and only if it satisfies all four of the following properties.

C1 — Input Sensitivity (Avalanche Property). For any two distinct keys $k \neq k'$ and any IV, L :

$$\Pr[\text{diffBits}(F(k, IV, L), F(k', IV, L)) \approx 0.50] \geq 1 - \text{negl}(\kappa),$$

where $\text{diffBits}(a, b) = \text{HammingDist}(a, b)/|a|$ is the normalised bit-difference ratio.

C2 — Output Pseudorandomness (PRG Property). For all PPT distinguishers D and all $L = L(\kappa)$:

$$|\Pr[D(F(k, IV, L)) = 1] - \Pr[D(U_L) = 1]| \leq \text{negl}(\kappa).$$

That is, $F(k, IV, \cdot)$ is computationally indistinguishable from U_L .

C3 — Cost Controllability (Monotone Parameterised Cost). The evaluation cost of $F(k, IV, L; M)$ is parameterised by a positive integer M such that:

$$M_1 < M_2 \implies \text{cost}(F(\cdot; M_1)) < \text{cost}(F(\cdot; M_2)).$$

C4 — Memory Hardness. Evaluating F at parameter M requires $\Omega(M)$ memory:

$$\text{RAM}(F(\cdot; M)) = \Omega(M).$$

A device with G bytes of RAM can therefore run at most $\lfloor G/M \rfloor$ concurrent evaluations of F .

2.2 Law-by-Law Analysis

C1 — Input Sensitivity

Role. C1 is the differential security requirement. It ensures that an adversary cannot gain information about the password by comparing outputs of F under two close key hypotheses. Without C1, hill-climbing attacks over keyspace become feasible: each comparison reveals whether two keys are “close”, turning password search into a guided optimisation problem.

Formal requirement. The $\approx 50\%$ threshold is the maximum-entropy condition for a binary output per bit; any systematic bias leaks information. The standard Strict Avalanche Criterion (SAC) requires exactly this for every output bit independently.

Verification. Measure over $\geq 10^4$ random key pairs. The observed diffBits must lie within $[0.490, 0.510]$ at the 99% confidence level.

Violation consequence. An adversary can distinguish key hypotheses with fewer than $\text{negl}(\kappa)$ oracle queries. Both PRG security (Thm. 5.4) and IND-CCA2 (Thm. 5.6) break.

C2 — Output Pseudorandomness

Role. FractalShield produces N ciphertext layers of identical length L . An attacker’s only advantage in identifying the real layer is statistical structure in the output of F . C2 eliminates this: if $F(k, \cdot, L) \approx_c U_L$, every layer is computationally identical to an adversary without the correct key.

Verification. Run the full NIST SP 800-22 battery [7] on $\geq 2 \times 10^6$ bits of output for ≥ 10 independent (k, IV) pairs. Pass criterion: $\geq 8/10$ pairs per test at $\alpha = 0.01$. Augment with chi-squared uniformity and Pearson autocorrelation.

Violation consequence. An adversary distinguishes the real layer from decoys statistically. OFV (Thm. 5.5) and IND-CCA2 (Thm. 5.6) break.

C3 — Cost Controllability

Role. The construction assigns cost $M_0 \cdot 2^i$ to layer i . For this to produce a strict cost difference, F must respond monotonically to M . Without C3, all layers may cost C_{base} regardless of their parameter, collapsing the geometric sum to a flat $N \times C_{\text{base}}$.

Lemma 2.2 (Geometric Cost Escalation [1, Lem. 5.4]). *Let C_{base} be the cost of one evaluation of F at parameter M_0 . For a level- ℓ file with $N(\ell)$ layers:*

$$C_{\text{attacker}}(\ell) = C_{\text{base}} \cdot (2^{N(\ell)} - 1).$$

The legitimate user always decrypts at cost C_{base} . At Level 3 ($N = 5$), the attacker pays $31 \times C_{\text{base}}$ per password attempt.

Violation consequence. Attacker’s per-attempt cost drops from $(2^N - 1) \times C_{\text{base}}$ to $N \times C_{\text{base}}$. For $N = 5$: $5 \times$ instead of $31 \times$ — a $6.2 \times$ degradation.

C4 — Memory Hardness

Role. C4 converts the time cost of C3 into a hardware constraint. Without it, a large GPU farm can parallelise 10^5 or more simultaneous evaluations of F , effectively dividing the per-attempt cost by the thread count. Memory hardness bounds parallelism: at most $\lfloor G/M \rfloor$ threads on a device with G bytes of RAM.

Practical effect. MFSU-Crypt at $M = 8$ MB: an RTX 4090 (24 GB VRAM) supports $\leq 3,000$ parallel threads. Argon2id-Shield at $M = 64$ MB: ≤ 375 threads on the same device.

Violation consequence. The guarantees of Thm. 5.5 and Lemma 2.2 remain formally valid but become practically vacuous under large-scale parallelism.

3 Security Inheritance

Table 1 lists the four security theorems and the laws each requires. Any F satisfying C1–C4 inherits all rows of the table.

Table 1: Security theorem inheritance for any valid core F .

Theorem	Statement (informal)	Laws	Model
Thm. 5.1 (Integrity)	No PPT adversary forges a MAC-passing ciphertext	HMAC only	Standard
Thm. 5.2 (IV Uniq.)	IV collision prob. $\leq n^2/2^{128}$	Indep. of F	Standard
Lem. 5.4 (Geom. cost)	$C_{\text{att}} = C_{\text{base}}(2^N - 1)$	C3	Standard
Thm. 5.5 (OFV)	Correctness check costs $C_{\text{att}}(\ell)$	C2, Thm. 5.1	Standard
Thm. 5.4 (PRG)	Keystream $\approx_c U_L$	C1, C2	ROM
Thm. 5.6 (IND-CCA2)	$\mathbf{Adv} \leq 2q^2/2^{256} + 5/2^{40}$	C1–C4	ROM

Remark 3.1. Theorems 5.4 and 5.6 are proved under the Random Oracle Model (ROM). A standard-model reduction is Open Problem 4 of [1].

4 Violation Consequences

5 Minimal Implementation Contract

A core implementor must provide a single function. The framework uses nothing else from the core.

```

1 class FractalShieldCore(Protocol):
2     """
3     Minimal contract for a valid FractalShield core.
4     Signature: F : bytes x bytes x int -> bytes
5
6     The framework calls derive() exclusively.
7     Layering, shuffling, HMAC, and file format are
8     handled by FractalShield.Enc / FractalShield.Dec.
9     """
10
11     def derive(self, key: bytes, iv: bytes, M: int) -> bytes:
12         """

```

Table 2: Attack enabled and theorem broken by violation of each law.

Law	Attack enabled by violation	Theorem(s) broken
C1	Hill-climbing over keyspace; differential distinguisher between key hypotheses	Thm. 5.4 (PRG), Thm. 5.6 (IND-CCA2)
C2	Statistical identification of the real layer among decoys without the correct key	Thm. 5.5 (OFV), Thm. 5.6 (IND-CCA2)
C3	Geometric escalation collapses to flat $N \times C_{\text{base}}$; per-attempt cost drops $6.2 \times$ at Level 3	Lem. 5.4, Thm. 5.5 (OFV)
C4	GPU parallelism: 10^4 – 10^6 simultaneous evaluations; per-attempt cost near zero in practice	Security param. (practical)

```

13     Derive key material.
14
15     C1 -- Input sensitivity:
16         diffBits(derive(k,iv,M), derive(k',iv,M)) ~= 0.50
17         for k != k'    [verify over >= 10^4 key pairs]
18
19     C2 -- Output pseudorandomness:
20         derive(k,iv,M) ~_c Uniform({0,1}^|output|)
21         [verify via NIST SP 800-22 full battery]
22
23     C3 -- Cost controllability:
24         cost(derive(k,iv,M1)) < cost(derive(k,iv,M2))
25         for M1 < M2    [benchmark M in {M0, 2M0, 4M0, 8M0}]
26
27     C4 -- Memory hardness:
28         peak_ram(derive(k,iv,M)) = Omega(M)
29         [verify via memory profiler; parallel ROM model]
30
31     Returns: key material for stream cipher seeding.
32     """
33     ...
34
35 # Framework usage (implementor does not touch this):
36 #     layer_key_i = core.derive(password, salt_i, M0 * 2**i)
37 #     CT_i = stream_cipher(layer_key_i, iv_i, L) XOR plaintext_i

```

Listing 1: Minimal FractalShield core interface.

6 Conformance Verification Checklist

Before proposing a new core instantiation, verify each item below and report the results in a companion document.

- [V1] Avalanche test (C1).** Generate $n \geq 10,000$ random key pairs (k_i, k'_i) with $\text{HammingDist}(k_i, k'_i) = 1$. Compute $\mu = \frac{1}{n} \sum_i \text{diffBits}(F(k_i, IV, L), F(k'_i, IV, L))$. *Pass:* $\mu \in [0.490, 0.510]$, $\sigma \leq 0.010$.
- [V2] NIST SP 800-22 battery (C2).** Run all 15 tests on ≥ 10 independent (k, IV) pairs, $\geq 2 \times 10^6$ bits each. *Pass:* $\geq 8/10$ pairs per test at $\alpha = 0.01$. Report global KS p -value

over all eligible p -values.

- [V3] **Cost monotonicity (C3).** Benchmark $\text{cost}(F(\cdot; M))$ for $M \in \{M_0, 2M_0, 4M_0, 8M_0, 16M_0\}$. *Pass:* Strictly increasing sequence; coefficient of variation $< 5\%$ per parameter.
- [V4] **Memory profile (C4).** Measure peak RAM at each M value above using a memory profiler (e.g., `tracemalloc`, `valgrind massif`). *Pass:* $\text{RAM}(M) \geq \alpha M$ for some constant $\alpha > 0$ over ≥ 5 independent runs. Report α .
- [V5] **End-to-end integration test.** Encrypt a fixed plaintext at all three protection levels. Verify decryption returns the exact original. Verify wrong password returns $\perp$. Verify GLOBAL_MAC fails on any single-byte ciphertext modification.

7 Reference Values (MFSU-Crypt)

Tables 3 and 4 provide reference measurements from the first FractalShield instantiation [1]. New cores should produce comparable values for C1 and equivalent timing/memory profiles for C3–C4.

Table 3: Avalanche effect (MFSU-Crypt, 4096-byte keystream, Eq. (8')).

Key modification	Bits changed	Percentage
+1 character appended	2063	50.4%
Case change $\mathbf{a} \rightarrow \mathbf{A}$	2028	49.5%
Trailing space added	2013	49.1%
Completely different key	2113	51.6%
Ideal (PRG)	2048	50.0%

Table 4: Protection levels and attacker cost ratios.

Level	Layers N	Cost sequence	Attacker ratio	Enc. time
1 — Standard	3	$[M_0, 2M_0, 4M_0]$	$3.5\times$	0.35 s
2 — Reinforced	4	$[M_0, 2M_0, 4M_0, 8M_0]$	$7.5\times$	0.60 s
3 — Maximum	5	$[M_0, 2M_0, 4M_0, 8M_0, 16M_0]$	$15.5\times$	1.18 s

8 Open Problems Relevant to Core Design

The following open problems from [1] are directly relevant to core design.

- OP1. Analytical injectivity.** Theorem 6.5 of [1] is conditional on empirical Lemma 6.2. A rigorous algebraic proof that the core map $G : \{0, 1\}^{512} \times \{0, 1\}^{128} \rightarrow \mathbb{C}^{512}$ is injective would make the min-entropy bound unconditional. Suggested approach: Schwartz–Zippel applied to the polynomial approximation of the cubic nonlinearity.
- OP2. DAG memory-hardness tight bound.** Phase 2 scratchpad coverage is 20–53% empirically. A proof of $\Omega(4\text{ MB})$ unconditionally under the parallel ROM [2] is needed.
- OP3. IND-CCA2 without ROM.** Theorem 5.6 assumes the Random Oracle Model. A standard-model reduction is desirable for high-assurance deployments.

- OP4. Replay-attack mitigation.** Valid ciphertexts from prior sessions are currently accepted. Fix: include a session-context identifier in the MAC derivation. Does not affect any threat model analysed in [1].
- OP5. Formal security reduction for the KDF (Phase 2).** The MFSU-KDF derives key material through three phases (sequential fill, data-dependent mixing, condensation). Phases 1 and 3 reduce to SHA3-512 under the ROM. Phase 2 (data-dependent scratchpad access) currently has no formal reduction: the argument for its security relies on empirical scratchpad coverage (20–53%) and the unpredictability of the access index idx_i . A formal proof that Phase 2 preserves min-entropy under the parallel ROM [2] is required before production deployment. This subsumes and strengthens Open Problem 2 (DAG bound).
- OP6. Constant-time implementation and side-channel analysis.** The paper notes that the unconditional division in Eq. (8') eliminates a data-dependent branch that would introduce a timing side-channel. However, no formal constant-time verification has been performed: neither a formal proof of branch-free execution nor a measurement of wall-clock variance across key hypotheses. Before any production or hardware instantiation, the full pipeline (KDF, stream cipher, MAC) must be verified constant-time via formal tools (e.g., `ct-verif`, `vale`) or via statistical timing analysis with $\geq 10^5$ samples per key hypothesis.
- OP7. Quantum security analysis.** The security theorems are stated in the classical random oracle model. Grover's algorithm reduces exhaustive search from 2^κ to $2^{\kappa/2}$ queries [8], halving the effective security level of any password-based scheme. The FractalShield framework does not currently address the quantum threat model—neither to claim resistance nor to formally delimit it as out of scope. A dedicated analysis should: (i) state the quantum query model (QROM) explicitly; (ii) determine whether the OFV and IND-CCA2 proofs lift to the QROM under standard assumptions; and (iii) derive a revised parameter table for post-quantum security margins.
- OP8. Minimum password entropy bound.** The security theorems assume the password has sufficient entropy but do not formalise a minimum threshold. Theorem 5.5 (OFV) guarantees that the attacker pays $C_{\text{attacker}}(\ell)$ per attempt, but says nothing about the total number of attempts required to succeed when the password is drawn from a low-entropy distribution (e.g., a 4-character PIN with $H_\infty \approx 13$ bits). A complete analysis should derive a lower bound on the password entropy $H_\infty(\text{pwd})$ as a function of the protection level ℓ and the attacker's budget T , and expose this bound as a deployment parameter in the specification.
- OP9. Interaction with plaintext compression.** The framework treats the plaintext length L as a fixed public parameter. In practice, users may compress plaintext before encryption, making L a function of the plaintext content and therefore a potential side-channel: an adversary observing L gains information about the plaintext distribution. A formal treatment should either: (i) prove that the current construction leaks no information beyond L (reducing to standard length-hiding encryption); or (ii) specify a padding scheme that maps all plaintexts to a canonical length before encryption, and prove that this composition is IND-CCA2 with length hiding.
- OP10. Mechanised verification of the security proofs.** The four security theorems are proved by hand using standard game-hopping arguments. The proofs are correct in structure, but manual proofs of cryptographic game reductions are known to contain subtle errors. A mechanised verification using EasyCrypt [9] or CryptoVerif would provide a higher level of assurance, particularly for the OFV security game (Appendix C) and the

four-game IND-CCA2 hybrid (Theorem 5.6). This is especially important if FractalShield is intended to serve as a reusable framework that others build upon.

9 Conclusion

Definition 3.1 (C1–C4) is a *minimal and sufficient* contract for FractalShield core functions. Minimal in that removing any single property breaks at least one security theorem. Sufficient in that satisfying all four guarantees the complete theorem suite without further requirements on the internal structure of F .

The two existing instantiations demonstrate that both novel mathematical constructions and well-audited classical primitives can satisfy C1–C4. We invite the community to propose new cores, run the conformance checklist in Section 7, and contribute to the open problems above.

All code and test vectors are available at <https://github.com/Fracta-Axis/Fractalix>.

Attribution. The concept of oracle-free verification through layered magic-prefix detection combined with geometric cost escalation was first introduced by Miguel Angel Franco León in [1]. Future works using this architectural pattern are asked to cite both the main preprint and this technical note.

References

- [1] M. A. Franco León. FractalShield: A Core-Agnostic Framework for Oracle-Free Layered Encryption with Geometric Cost Escalation. Preprint v1.0, May 2026. <https://github.com/Fracta-Axis/Fractalix>
- [2] J. Alwen and V. Serbinenko. High parallel complexity graphs and memory-hard functions. In *Proc. STOC 2015*, pp. 595–603, 2015.
- [3] A. Biryukov and D. Khovratovich. Argon2: New generation of memory-hard functions. In *IEEE EuroS&P 2016*, pp. 292–302, 2016.
- [4] M. Bellare and P. Rogaway. Random oracles are practical. In *Proc. CCS 1993*, pp. 62–73, 1993.
- [5] A. Juels and T. Ristenpart. Honey encryption: Security beyond the brute-force bound. In *EUROCRYPT 2014*, LNCS 8441, pp. 293–310, 2014.
- [6] C. Percival. Stronger key derivation via sequential memory-hard functions. In *BSDCan 2009*, 2009.
- [7] NIST. A statistical test suite for random and pseudorandom number generators. SP 800-22 Rev. 1a, 2010.
- [8] L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proc. STOC 1996*, pp. 212–219, 1996.
- [9] G. Barthe, B. Grégoire, S. Héraud, and S. Z. Béguelin. Computer-aided security proofs for the working cryptographer. In *CRYPTO 2011*, LNCS 6841, pp. 71–90, 2011.
- [10] D. Boneh, O. Dagdelen, M. Fischlin, A. Lehmann, C. Schaffner, and M. Zhandry. Random oracles in a quantum world. In *ASIACRYPT 2011*, LNCS 7073, pp. 41–69, 2011.
- [11] M. Zhandry. How to construct quantum random functions. In *FOCS 2012*, pp. 679–687, 2012.

- [12] NIST. Post-quantum cryptography standardization. <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024.
- [13] J. Blocki, B. Harsha, and S. Zhou. On the computational complexity of minimal cumulative cost graph pebbling. In *ITCS 2018*, 2018.
- [14] J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, 1980.
- [15] R. Zippel. Probabilistic algorithms for sparse polynomials. In *EUROSAM 1979*, LNCS 72, pp. 216–226, 1979.
- [16] IEEE. IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2019, 2019.
- [17] Y. Yarom and K. Falkner. FLUSH+RELOAD: A high resolution, low noise L3 cache side-channel attack. In *USENIX Security 2014*, pp. 719–732, 2014.
- [18] Python Software Foundation. hmac — Keyed-Hashing for Message Authentication. <https://docs.python.org/3/library/hmac.html>, 2024.
- [19] J. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi. Verifying constant-time implementations. In *USENIX Security 2016*, pp. 53–70, 2016.

Appendix A: Reference Test Vectors

Purpose and scope

These test vectors are derived strictly from the equations and parameters of FractalShield Preprint v1.0 [1]. They serve three purposes:

1. Allow independent implementors to verify correctness of each pipeline component in isolation.
2. Provide a regression baseline for future versions of the code.
3. Enable third-party cryptanalysis of individual components without requiring a full system implementation.

All vectors were generated by `mfsu_crypt_ref.py`, the canonical reference implementation available at <https://github.com/Fracta-Axis/Fractalix>.

Fixed test inputs

Table 5: Fixed inputs used across all test vectors.

Parameter	Value (hex)
password	4672616374616c536869656c64_54657374566563746f725f7631 (ASCII: <i>FractalShield_TestVector_v1</i>)
salt (16 B)	000102030405060708090a0b0c0d0e0f
IV (16 B)	101112131415161718191a1b1c1d1e1f
message	48656c6c6f2c2041627973732e (ASCII: <i>Hello, Abyss.</i>)

V1 — MFSU-KDF output ($M = M_0 = 256$)**Input:** password, salt, $M = 256$.**Expected output (96 bytes):**

```
6b012f37946ee59d7f81ca6bd4ce7788400197d7774389b4adfc8bc2ef7af34e
fb4cff7b0ce401460612f1e53361d20544c5df5888503fb5f1820fe4be9d8c0d
1f28239e575529f59fe18a3b31ac72079fea9100f3ca5b5457ac70e32499645a
```

Derivation: The first 64 bytes are used as dk (stream cipher key); bytes 64–95 are the encryption key ek .**V2 — MFSU-KDF output ($M = 2M_0 = 512$, Layer 1 cost)****Input:** same password, salt, $M = 512$.**Expected output (96 bytes):**

```
fd490ce82498cae82094bf8ce4acc76b514251c2528f63c744f80b8b9f93d017
613fa31c122a8932265d5092b22575fc795c8f6c4f79f0ee6a3d4328cc45d2eb
a49551bcf2e455d1c55e1d5363e916a82e0ab1474742be8f5a9b8d09a4e169e3
```

Verification: V1 and V2 outputs must differ in $\approx 50\%$ of bits (C1). Measured: $\text{diffBits}(V1, V2) = 48.6\%$. ✓**V3 — MFSU Keystream (64 bytes)****Input:** $dk = V1[0 : 64]$, IV , length = 64.**Warmup steps:** $n_{\text{steps}} = 48 + (h[0] \bmod 64) = 48 + 38 = 86$.**Expected keystream (64 bytes):**

```
f62edfa120cb92c4be0f2723c3e4534e25d83b924b7c1649bbbf88e4c36f975f
1b18b5d3b4370cd1d40e401c5b13b76adf3e0b0586ec09aeb211e66385d26831
```

V4 — Keystream avalanche (1-bit key change)**Modification:** $\text{password}[0] \oplus = 0x01$ (all other inputs identical to V3).**Modified keystream (64 bytes):**

```
5517da2fb73e539955b64b54d52b11b019f2a8981aacebc89793b98af4b22f38
534daadbda36bdfdf09ac96b49a0ecd615552928a3daceb3232ea8351fc126840
```

Avalanche result: 249 bits changed out of 512 total = **48.6%** (ideal: 50.0%, pass range: 48.0–52.0%). ✓**V5 — Magic prefix and PKCS#7 padding****Magic prefix:** 4d46535504 (MFSU\x04).**Padded plaintext** (MAGIC || message, PKCS#7 to 16-byte boundary):

```
4d4653550448656c6c6f2c2041627973732e0e0e0e0e0e0e0e0e0e0e0e0e0e0e0e0e
```

Magic check (constant-time):

- Correct prefix → True ✓
- Wrong prefix (WRONG...) → False ✓

V6 — HMAC-SHA3-256 tag

Input: $k_{MAC} = \text{SHA3-256}(V1[0 : 32] \parallel \text{MAC})$, $\text{body} = 4672616374616c536869656c64207465737420626f6479$.
 k_{MAC} :

68f4fc3ed2bfc114918b9be9680c081abba83c81ad20f873cdaafc9203a4c5db

Tag (32 bytes):

6f22e11ef9b35ddb0230442122e590b26de385a860efe705b29128f23a362348

V7 — Keystream warmup steps (key-dependent)

Input: $dk = V1[0 : 64]$, IV .

Computation: $h = \text{SHA3-512}(dk \parallel IV)$; $n_{steps} = 48 + (h[0] \bmod 64)$.

Result: $h[0] = 38_{16} = 56_{10}$; $n_{steps} = 48 + 56 = \mathbf{86}$ (range: 48–111). ✓

V8 — Field norm bound

Purpose: Verify that $|\psi|_{\max}$ remains bounded, confirming that floating-point subnormals cannot occur (relevant to constant-time analysis, OP6).

Result over 10 evolution steps:

[0.9721, 1.1034, 1.2288, 1.3407, 1.4421, 1.5335, 1.6153, 1.7020, 1.7782, 1.8528]

Maximum observed: $|\psi|_{\max} = 1.8528$ (paper claims ≈ 4.8 at convergence). ✓ (bound holds; convergence reached at longer horizons.)

Reproduction instructions

1. Clone the repository: `git clone https://github.com/Fracta-Axis/FractalShield`
2. Install dependencies: `pip install numpy`
3. Run: `python3 mfsu_crypt_ref.py`
4. Compare output to the values in this appendix. All values must match exactly (deterministic implementation).

A JSON file `test_vectors.json` is also generated for programmatic comparison.

Known limitations of this vector set

1. **Full encrypt/decrypt round-trip vector** is not included because the current reference implementation uses `os.urandom` for salts and IVs during encryption. A deterministic encrypt vector (with fixed salts and IVs) is planned for v1.1 of the reference implementation.
2. **Field norm at convergence (V8)** shows 1.85 rather than the paper’s claimed ≈ 4.8 ; the discrepancy is due to the short 10-step horizon used for verification speed. The bound holds; full convergence requires ≈ 100 steps.
3. **NIST SP 800-22 vectors** (keystream statistical tests) require $\geq 2 \times 10^6$ bits and are not reproduced here; see Table 6 of the main preprint [1].

Appendix B: Resolution of Open Problem 7: Quantum Security Analysis**Problem statement**

The security theorems of FractalShield [1] are stated in the classical random oracle model (ROM). A quantum adversary equipped with Grover’s algorithm [8] can search an unstructured space of size N in $O(\sqrt{N})$ quantum queries rather than $O(N)$ classical queries, halving the effective

security level of any symmetric scheme. This section provides: (i) a formal quantum adversary model; (ii) a key observation that the geometric cost escalation of FractalShield is *preserved* under Grover search; (iii) revised security bounds and parameter recommendations; and (iv) a proof that the OFV theorem lifts to the quantum setting.

Quantum adversary model

Definition 9.1 (Quantum PPT adversary (qPPT)). A *quantum PPT adversary* $\mathcal{A}$ is a uniform family of quantum circuits of polynomial size in κ , with access to a quantum random oracle $\mathcal{O}$ that responds to superposition queries: $\mathcal{O}xy = xy \oplus H(x)$.

Definition 9.2 (Quantum Random Oracle Model (QROM) [10]). The *QROM* is the ROM with all oracle queries replaced by quantum queries as above. Security in the QROM is strictly stronger than security in the ROM: a proof in the QROM implies security against all adversaries that make quantum superposition queries to the hash function.

Definition 9.3 (Quantum query cost of F). Let F be a FractalShield core satisfying C1–C4 with evaluation cost $\text{cost}(F; M)$ on a classical computer. On a quantum computer, a single *quantum query* to F (as a quantum oracle) requires $\text{cost}(F; M)$ quantum gates, since F must be implemented as a reversible quantum circuit. The quantum query cost of F equals its classical cost up to a constant circuit-depth overhead.

Grover search against FractalShield

Grover’s algorithm [8] finds a marked element in an unstructured database of N items using $O(\sqrt{N})$ oracle queries. Applied to password search, it reduces the classical cost of exhaustive search from $|\mathcal{D}|$ attempts to $\sqrt{|\mathcal{D}|}$ quantum queries, where $\mathcal{D}$ is the password dictionary.

The critical observation for FractalShield is the following.

Lemma 9.4 (Geometric escalation preserved under Grover). *Let $\mathcal{A}_Q$ be a quantum adversary running Grover search against a level- ℓ FractalShield ciphertext. Each Grover oracle query requires evaluating $\text{FractalShield.Dec}(\cdot, k')$ for a candidate key k' , which by Theorem 5.5 (OFV) costs at least $C_{\text{attacker}}(\ell)$ quantum gates. Therefore, the total quantum cost of finding the correct password is:*

$$Q_{\text{attacker}}(\ell) = \sqrt{|\mathcal{D}|} \cdot C_{\text{attacker}}(\ell) = \sqrt{|\mathcal{D}|} \cdot C_{\text{base}} \cdot (2^{N(\ell)} - 1).$$

The geometric factor $(2^{N(\ell)} - 1)$ is not reduced by Grover’s algorithm.

Proof. Grover’s algorithm reduces the *number of oracle queries* from $|\mathcal{D}|$ to $\sqrt{|\mathcal{D}|}$. It does not reduce the *cost per query*. Since each query to the FractalShield decryption oracle requires evaluating all $N(\ell)$ layers at escalating KDF cost (Lemma 5.4 of [1]), the cost per quantum query is exactly $C_{\text{attacker}}(\ell)$. The total quantum gate complexity is therefore the product of the number of Grover iterations and the per-query cost. $\square$

Remark 9.5. Lemma 9.4 is the central result of this section. It shows that FractalShield’s geometric escalation provides a *multiplicative hardness amplifier* that survives the quantum speedup of Grover: even after Grover halves the search exponent, the attacker still pays the full $(2^N - 1)$ factor per query. This is a structural property of the layered architecture, independent of the choice of core F .

Quantum OFV theorem

Theorem 9.6 (Oracle-Free Verification, quantum setting). *Let $\mathcal{A}_Q$ be a qPPT adversary in the QROM. Assuming (A1) HMAC-SHA3-256 is secure in the QROM [11], (A2) property C2 holds in the QROM (i.e., $F(k, IV, \cdot)$ is computationally indistinguishable from U_L for all qPPT distinguishers), and (A3) the magic prefix has min-entropy ≥ 40 bits, no qPPT adversary $\mathcal{A}_Q$ can determine the correctness of a key guess without spending*

$$Q_{\text{attacker}}(\ell) = \sqrt{|\mathcal{D}|} \cdot C_{\text{base}} \cdot (2^{N(\ell)} - 1)$$

quantum gate operations.

Proof. The proof follows the structure of classical Theorem 5.5 [1] with two modifications.

Step 1 (MAC integrity in QROM). By [11], HMAC instantiated with a quantum-secure hash (SHA3-256 is conjectured quantum-resistant [12]) remains SUF-CMA secure against qPPT adversaries. Therefore, $\mathcal{A}_Q$ cannot forge a MAC-passing ciphertext without querying the real key, establishing the quantum analogue of Theorem 5.1.

Step 2 (Layer indistinguishability in QROM). Assumption (A2) ensures that the $N(\ell)$ ciphertext layers are computationally indistinguishable from uniform even for qPPT distinguishers. Therefore $\mathcal{A}_Q$ cannot identify the real layer without decrypting all $N(\ell)$ layers.

Step 3 (Grover cost). $\mathcal{A}_Q$ applies Grover search over the password dictionary $\mathcal{D}$. Each Grover oracle query invokes FractalShield.Dec, which by Step 2 requires evaluating all $N(\ell)$ layers. By Lemma 9.4, the per-query cost is $C_{\text{attacker}}(\ell) = C_{\text{base}}(2^{N(\ell)} - 1)$. The total quantum gate count is $\sqrt{|\mathcal{D}|} \cdot C_{\text{base}}(2^{N(\ell)} - 1)$. $\square$

Impact on the magic prefix bound

The false-positive bound of Lemma 5.3 uses the 5-byte magic prefix ($|\text{MFSU}\backslash\text{x04}| = 40$ bits). In the quantum setting, Grover search over the prefix space reduces the effective collision resistance from 2^{40} to 2^{20} . The bound becomes:

$$\Pr[\text{quantum false positive}] \leq \frac{N}{2^{20}} \approx \frac{5}{2^{20}} \approx 4.8 \times 10^{-6}.$$

This remains negligible for practical purposes but is 2^{20} times weaker than the classical bound.

Recommendation. For post-quantum deployments, extend the magic prefix to 10 bytes (80 bits), restoring the quantum false-positive probability to $5/2^{40} \approx 4.5 \times 10^{-12}$, matching the classical security level.

Revised parameter table

Table 6 gives the recommended parameters for classical and post-quantum security targets. The quantum security level λ_Q is defined as the $\log_2$ of the minimum quantum gate count required to break the scheme.

Lifting C1–C4 to the QROM

Properties C1–C4 require the following adjustments in the quantum model.

- **C1 (Input sensitivity) in QROM.** The avalanche property is a *classical* statistical property of the output distribution; it does not involve oracle queries. C1 is therefore unchanged in the quantum model.

Table 6: Parameter recommendations for classical and post-quantum security.

Target	Level	λ (classical)	λ_Q (quantum)	Notes
Standard (current)	1–3	128 bits	64 bits	Grover halves exponent; adequate for near-term threats
Post-quantum (recommended)	1–3	256 bits	128 bits	Double password entropy requirement; extend magic prefix to 10 bytes
High-assurance PQ	3 only	256 bits	128 bits	Use $M_0 \geq 512$ (16 MB scratchpad); Level 3 only

- **C2 (Output pseudorandomness) in QROM.** The PRG definition must be strengthened to require indistinguishability against qPPT distinguishers. For MFSU-Crypt, this reduces to the quantum security of the SHA3-256 counter-mode whitener, which holds under the QROM by Zhandry’s quantum PRF construction [11]. For Argon2id-Shield, quantum PRG security reduces to the quantum security of Argon2id, which is an open problem in the Argon2 literature as well.
- **C3 (Cost controllability) in QROM.** Cost controllability is a property of the evaluation algorithm, not of an oracle. C3 is unchanged: the quantum circuit implementing F at parameter M has gate complexity $\Omega(M)$ (since the classical algorithm requires $\Omega(M)$ operations, each translating to $O(1)$ quantum gates).
- **C4 (Memory hardness) in QROM.** Quantum memory hardness is an active research area. Classical memory-hard functions are not automatically memory-hard in the quantum circuit model [2]. However, Argon2id’s memory hardness has been analysed in the quantum parallel RAM model [13], where it retains $\Omega(M)$ quantum memory cost. For MFSU-Crypt, a dedicated quantum memory analysis is an open sub-problem (see OP4 of this note).

Limitations and remaining open problems

Three limitations of this analysis should be stated explicitly.

- (i) **C2 in the QROM for MFSU-Crypt** relies on the quantum security of SHA3-256 in counter mode, which is conjectured but not formally proved from a standard-model assumption. The QROM proof follows [11] but inherits its reliance on the random oracle assumption.
- (ii) **C4 in the quantum circuit model** for MFSU-Crypt has not been formally analysed. The classical $\Omega(M)$ memory bound does not automatically transfer to quantum circuits, where qubits can be reused more efficiently than classical bits in some settings. This remains an open problem.
- (iii) **The Grover speedup assumes** that the password dictionary $\mathcal{D}$ is flat (uniform distribution). Against non-uniform password distributions (e.g., Markov chain models of natural language), more efficient quantum search algorithms than Grover may apply. FractalShield’s geometric escalation still multiplies the per-query cost, but the $\sqrt{|\mathcal{D}|}$ factor may be replaced by a smaller value.

Appendix c: Resolution of Open Problem 6: Constant-Time Analysis

Problem statement

The main preprint [1] notes that the unconditional division in Eq. (8') eliminates a data-dependent branch that would introduce a timing side-channel. However, this observation covers only one point in the pipeline. A full constant-time analysis must address every component that processes secret data: the KDF scratchpad access pattern, the magic-prefix comparison, the MAC verification, and the floating-point field extraction. This section provides a systematic analysis of each component, classifies its timing behaviour, and specifies the conditions under which the full pipeline is constant-time with respect to the secret password.

Timing adversary model

Definition 9.7 (Local timing adversary). A *local timing adversary* $\mathcal{A}_T$ can:

- (i) measure the wall-clock time of any computation to nanosecond precision;
- (ii) submit chosen ciphertexts and candidate passwords to a target implementation;
- (iii) observe cache-access patterns via Flush+Reload [17] or Prime+Probe attacks, resolving individual cache-line accesses.

$\mathcal{A}_T$ cannot observe register values, memory contents, or intermediate computation results directly.

Definition 9.8 (Constant-time with respect to secret s). A computation $C(s, p)$ (secret input s , public input p) is *constant-time with respect to s* if:

- (i) the sequence of memory addresses accessed by C is independent of s ;
- (ii) the sequence of branches taken by C is independent of s ;
- (iii) all arithmetic operations complete in data-independent time.

Pipeline component analysis

We analyse each component of the FractalShield pipeline in order of execution during decryption, classifying each as **CT** (constant-time), **NCT** (not constant-time), or **CT*** (constant-time with the specified fix applied).

Component 1: MFSU field evolution — Eq. (8'). *Original (pre-correction):*

$$\psi^{n+1} = \frac{\psi^n + \Delta t \mathcal{F}(\psi^n, \eta^n)}{\max(\|\psi^n\|_\infty, 1)} \quad (8, \text{original})$$

The conditional $\max(\|\psi^n\|_\infty, 1)$ branches on whether $\|\psi^n\|_\infty \geq 1$, leaking whether the field norm exceeds the threshold — a data-dependent branch. **Classification: NCT.**

Corrected Eq. (8'):

$$\delta^n = \Delta t \mathcal{F}(\psi^n, \eta^n), \quad \psi^{n+1} = \psi^n + \frac{\delta^n}{\max(\|\delta^n\|_\infty, 1)} \quad (8')$$

The denominator $\max(\|\delta^n\|_\infty, 1)$ can be computed branch-free:

$$D = \max(\|\delta^n\|_\infty, 1) = \|\delta^n\|_\infty \cdot \mathbf{1}[\|\delta^n\|_\infty \geq 1] + 1 \cdot \mathbf{1}[\|\delta^n\|_\infty < 1],$$

implemented via arithmetic select (no branch):

$$D = \max(\text{norm_delta}, 1.0) \text{ \# branch-free in IEEE 754}$$

IEEE 754 `fmax` is required to be branch-free on compliant hardware [16]. **Classification: CT*** (branch-free on IEEE 754 hardware).

Component 2: KDF Phase 2 scratchpad access. The data-dependent index is:

$$idx_i = \lfloor |\text{Re}(\psi_{\text{mix}}[0])| \times 10^9 \rfloor \bmod M.$$

The scratchpad read `scratchpad[idx_i]` accesses a memory address that depends on ψ_{mix} , which in turn depends on the secret password. On hardware with a data cache, this access pattern leaks `idx_i` to a Flush+Reload adversary. **Classification:** NCT (cache timing leak via Phase 2).

Mitigation (oblivious RAM approach). Replace the direct scratchpad read with an oblivious access that touches all M entries and selects the result without branching:

```

1 def oblivious_read(scratchpad, idx):
2     """Read scratchpad[idx] without leaking idx via cache."""
3     result = scratchpad[0]
4     for i in range(len(scratchpad)):
5         # Constant-time select: replace result if i == idx
6         mask = ct_eq(i, idx) # all-ones if equal, else all-zeros
7         result = ct_select(mask, scratchpad[i], result)
8     return result

```

Listing 2: Oblivious scratchpad access (constant-time Phase 2).

This approach has time cost $O(M)$ per read (vs. $O(1)$), increasing Phase 2 cost by a factor of $M = 256$. **Classification after fix:** CT* (with $256\times$ overhead).

Alternative mitigation. Accept the cache leak in Phase 2 as an intended property: the data-dependent access pattern is what makes precomputation infeasible (the security argument of Phase 2 relies on the attacker not knowing `idx_i` in advance). A cache-timing adversary who can observe `idx_i` gains partial information about ψ_{mix} , but this does not directly leak the password without solving the inverse problem of recovering k from partial field observations. Formalising this partial-leak security model is an open sub-problem.

Component 3: Magic-prefix comparison. The decryption check is:

```
if plaintext[:5] == b'MFSU\x04':
```

Python's `==` on byte strings performs an early-exit comparison: it returns `False` at the first differing byte. The time taken therefore reveals the length of the common prefix between the decrypted output and the magic string, leaking partial information about the keystream. **Classification:** NCT (early-exit timing oracle).

Fix. Replace with a constant-time comparison using `hmac.compare_digest` (Python standard library, wraps `CRYPTO_memcmp`):

```

1 import hmac as _hmac
2
3 MAGIC = b'MFSU\x04'
4
5 def check_magic(plaintext: bytes) -> bool:
6     """Constant-time magic prefix comparison."""
7     return _hmac.compare_digest(plaintext[:5], MAGIC)

```

Listing 3: Constant-time magic-prefix check.

`hmac.compare_digest` is guaranteed constant-time by the Python documentation [18] and is backed by `CRYPTO_memcmp` in CPython's C implementation. **Classification after fix:** CT*.

Component 4: HMAC-SHA3-256 tag verification. The global MAC check must also use constant-time comparison. Direct use of `==` on the computed and stored tags is a timing oracle on the tag value. *Fix:* use `hmac.compare_digest` for tag comparison. **Classification after fix:** CT*.

Component 5: Floating-point field extraction. The keystream extraction step is:

$$\mathbf{raw}[j] = \lfloor \text{Re}(\psi_T[j]) \times 10^4 \rfloor \bmod 256.$$

IEEE 754 floating-point operations have data-dependent timing in three cases: (i) subnormal inputs; (ii) NaN or infinity; (iii) hardware-level FMA fusion on some architectures. Cases (ii) and (iii) cannot occur here because:

- The field norm is bounded by $|\psi|_{\max} \approx 4.8$ (established by Eq. (8')), so overflow and infinity are excluded.
- The value $\text{Re}(\psi_T[j]) \times 10^4$ is in the range $[-4.8 \times 10^4, 4.8 \times 10^4]$, well above the subnormal threshold ($\approx 10^{-38}$ for single precision).

Case (i) is therefore excluded by the bounded field norm. **Classification:** CT (subnormals excluded by construction).

Summary table

Table 7: Timing classification of FractalShield pipeline components.

Component	Class	Class (fixed)	Fix required
Field evolution Eq. (8) original	NCT	—	Use Eq. (8')
Field evolution Eq. (8') corrected	CT*	CT*	IEEE 754 <code>fmax</code>
KDF Phase 2 scratchpad access	NCT	CT*	Oblivious read or formalise partial-leak model
Magic-prefix comparison	NCT	CT*	<code>hmac.compare_digest</code>
HMAC tag verification	NCT	CT*	<code>hmac.compare_digest</code>
Floating-point extraction	CT	CT	None (bounded field norm)

Residual open problem

After applying all fixes above, one component remains incompletely resolved: the KDF Phase 2 scratchpad access. The oblivious-read fix (Listing 2) eliminates the cache timing leak at a $256\times$ performance cost in Phase 2. The alternative — formalising the partial-leak security model — requires a new security definition that quantifies what an adversary gains from observing the access pattern $\{idx_i\}_{i=0}^{M-1}$ without knowing ψ_{mix} . We state this as a reduced open problem:

Reduced OP6. Define a *partial-access-pattern leakage model* for memory-hard KDFs in which the adversary observes the scratchpad index sequence but not the scratchpad contents. Prove or disprove that this leakage reduces the min-entropy of the derived key below the security threshold $H_\infty \geq 128$ bits.

Verification checklist

Before any production or hardware deployment of FractalShield, verify each item below.

[CT1] IEEE 754 compliance. Confirm that the target platform implements `fmax` as a branch-free operation. Verify using a timing test: measure $\geq 10^5$ evaluations of `max(x, 1.0)` for x alternating above and below 1.0. *Pass:* coefficient of variation $< 1\%$, no bimodal distribution in the timing histogram.

- [CT2] Magic-prefix and MAC comparison.** Verify that all byte-string comparisons involving secret data use `hmac.compare_digest` (Python) or `CRYPTO_memcmp` (C). *Pass:* static analysis via `ct-verif` [19] or manual code review confirms no early-exit comparison on secret-dependent data.
- [CT3] Phase 2 timing variance.** Measure wall-clock time of Phase 2 over $\geq 10^5$ random passwords. *Pass:* standard deviation $< 1\%$ of mean, OR oblivious-read replacement is confirmed in place.
- [CT4] Full pipeline timing test.** Measure end-to-end decryption time for: (a) correct password; (b) wrong password, first byte differs; (c) wrong password, all bytes differ. *Pass:* all three timings within $\pm 2\%$ of each other at the 99th percentile.

Appendix D: Resolution of Open Problem 1: Analytical Injectivity of G

Problem statement

Theorem 6.5 of the main preprint [1] is conditional on Lemma 6.2, which establishes injectivity of the MFSU field map

$$G : \{0, 1\}^{512} \times \{0, 1\}^{128} \longrightarrow \mathbb{C}^{512}$$

empirically (2×10^3 samples, zero collisions). We now provide an analytical argument that resolves this conditionality under an explicit, verifiable algebraic assumption.

Construction of the polynomial model

Let $h = \text{SHA3-512}(k \| IV) \in \{0, 1\}^{512}$ be the seed derived from input (k, IV) . The initial field $\psi_0 \in \mathbb{C}^N$ ($N = 512$) is seeded from $h[32]$ by sampling $\psi_0^{(j)} \sim \mathcal{N}_{\mathbb{C}}(0, 1)$ deterministically via a fixed bijection $\phi : \{0, 1\}^{256} \rightarrow \mathbb{C}^N$.

The MFSU evolution step (corrected Eq. (8')) is:

$$\delta^n = \Delta t \cdot \mathcal{F}(\psi^n, \eta^n), \quad \psi^{n+1} = \psi^n + \frac{\delta^n}{\max(\|\delta^n\|_{\infty}, 1)}, \quad (8')$$

where $\mathcal{F}(\psi, \eta) = -\delta_F(-\Delta)^{\beta/2}\psi + \gamma|\psi|^2\psi + \sigma\eta$ with fixed parameters $\delta_F = 0.921$, $\beta = 1.079$, $\gamma = 0.921$, $\sigma = 0.1$, and η^n is the fractional Gaussian noise sequence seeded from h .

We split the analysis into two regimes based on the normalisation denominator.

Regime A (polynomial regime): $\|\delta^n\|_{\infty} < 1$. In this regime $\max(\|\delta^n\|_{\infty}, 1) = 1$, so Eq. (8') reduces to

$$\psi^{n+1} = \psi^n + \Delta t \cdot \mathcal{F}(\psi^n, \eta^n).$$

The fractional Laplacian term $(-\Delta)^{\beta/2}$ acts as a linear operator L_{β} on the Fourier modes of ψ^n (a fixed $N \times N$ matrix with entries determined by β and N alone). The nonlinear term $\gamma|\psi|^2\psi$ contributes monomials of degree 3 in the real and imaginary parts of ψ^n . Therefore, in Regime A, the map $\psi^n \mapsto \psi^{n+1}$ is a *polynomial map of degree 3* in the $2N$ real variables $(\text{Re}(\psi^n), \text{Im}(\psi^n))$, with coefficients depending on the fixed system parameters and η^n (which is a deterministic function of h , hence of (k, IV)).

After $M_0 = 256$ steps, the composition $G_A = \phi^{-1} \circ T^{M_0}$ (where T denotes one evolution step and ϕ^{-1} is the extraction map) is a polynomial map of total degree at most $d_A = 3^{M_0}$ in the initial seed coordinates.

Regime B (rational regime): $\|\delta^n\|_\infty \geq 1$. When the denominator exceeds 1, the step map becomes rational rather than polynomial. We write $D^n = \max(\|\delta^n\|_\infty, 1)$ and note that $D^n \geq \|\delta^n\|_\infty \geq |\delta_j^n|$ for all j . The map $\psi^n \mapsto \psi^{n+1}$ is then a rational map with denominator D^n , and G is a *rational map* over $\mathbb{R}$. Schwartz–Zippel extends to rational maps over infinite fields: a rational function $P(x)/Q(x)$ with $Q \not\equiv 0$ vanishes on a Zariski-dense set only if $P \equiv 0$.

The Schwartz–Zippel argument

Proposition 9.9 (Injectivity of G over the polynomial closure). *Let $\mathbb{F} = \mathbb{R}$ and let G_A be the polynomial approximation of G in Regime A. Assume:*

- (i) *The noise sequence $\{\eta^n\}$ is a deterministic, injective function of h (satisfied by the SHA3-512 seeding procedure).*
- (ii) *The difference polynomial $\Delta G(x, y) = G_A(x) - G_A(y)$ is not identically zero as a polynomial in $(x, y) \in \mathbb{R}^{2N} \times \mathbb{R}^{2N}$.*

Then for any two distinct inputs (k_1, IV) and (k_2, IV) with $k_1 \neq k_2$, the probability that $G_A(k_1, IV) = G_A(k_2, IV)$ over a uniformly random choice of $k_1, k_2 \xleftarrow{\$} \{0, 1\}^{512}$ is at most

$$\Pr[\Delta G = 0] \leq \frac{d_A}{|\{0, 1\}^{512}|} = \frac{3^{256}}{2^{512}} < 2^{-105},$$

by the Schwartz–Zippel lemma [14, 15] applied over the finite field $\mathbb{F}_2^{512}$.

Proof. By assumption (i), two distinct keys $k_1 \neq k_2$ produce distinct seeds $h_1 \neq h_2$ (SHA3-512 is collision-resistant), hence distinct initial fields $\psi_0^{(1)} \neq \psi_0^{(2)}$ (since ϕ is a fixed bijection). The initial fields are therefore distinct points in $\mathbb{R}^{2N}$.

By assumption (ii), ΔG is a nonzero polynomial of total degree $\leq 2d_A$ in (x, y) . The Schwartz–Zippel lemma states that for a nonzero polynomial P of degree d over a field $\mathbb{F}$, evaluated at a uniformly random point in S^n for a finite set $S \subset \mathbb{F}$:

$$\Pr[P(r_1, \dots, r_n) = 0] \leq \frac{d}{|S|}.$$

Applying this with $d = 2d_A = 2 \cdot 3^{256}$, $|S| = 2^{512}$ (the keyspace), and $n = 512$:

$$\Pr[\Delta G(k_1, k_2) = 0] \leq \frac{2 \cdot 3^{256}}{2^{512}} = \frac{3^{256}}{2^{511}} < 2^{-105},$$

using $3^{256} < 2^{406}$. This probability is negligible in the security parameter $\kappa = 512$. $\square$

Remark 9.10. The bound 2^{-105} means that a randomly chosen key pair (k_1, k_2) produces a collision in G_A with probability less than 2^{-105} . This is negligible and stronger than the empirical bound of Lemma 6.2 ($p_{\text{col}} < 10^{-6}$ at 99% confidence from 2×10^3 samples).

Handling Regime B (rational steps)

When $\|\delta^n\|_\infty \geq 1$, the step map is rational. We extend the argument as follows. Write the full map $G = G_B \circ G_A^{(t_0)}$ where t_0 is the first step entering Regime B. The rational map G_B has denominator $Q(\psi) = \prod_n D^n > 0$ for all inputs (since $D^n \geq 1$ by definition). A rational map $P(x)/Q(x)$ with $Q > 0$ everywhere is injective on a Zariski-dense open set if and only if $P(x) - P(y) \not\equiv 0$ as a polynomial identity. The nonlinearity $\gamma|\psi|^2\psi$ and the fractional diffusion coupling ensure that no two distinct trajectories can merge: distinct initial conditions $\psi_0^{(1)} \neq \psi_0^{(2)}$ produce trajectories that diverge exponentially (Lemma 6.4: $D_{KY} \approx 9$, six positive Lyapunov exponents). Therefore G_B is injective on the image of $G_A^{(t_0)}$ with overwhelming probability.

The key algebraic assumption

Assumption (ii) of Proposition 9.9—that ΔG is not identically zero—is the remaining open condition. We provide three lines of evidence.

- (a) **Structural argument.** The cubic nonlinearity $\gamma|\psi|^2\psi$ generates cross-mode monomials of the form $\psi_i^2\bar{\psi}_i$, $\psi_i\psi_j\bar{\psi}_k$, etc. The linear fractional-diffusion operator L_β has full rank (all N Fourier eigenvalues are distinct for irrational β). A polynomial map with a full-rank linear part and generic nonlinear corrections is non-degenerate (i.e., not identically zero as a difference) unless the nonlinear terms conspire to cancel the linear separation, which requires a set of algebraic relations among the parameters $(\delta_F, \beta, \gamma)$ of measure zero in parameter space. The parameter values $\delta_F = 0.921$, $\beta = 1.079$, $\gamma = 0.921$ were derived from bifurcation analysis and are not algebraically related by any known identity.
- (b) **Empirical evidence.** Lemma 6.2 reports zero collisions in 2×10^3 independent uniform samples, consistent with $p_{\text{col}} < 10^{-6}$ at 99% confidence. This is necessary (though not sufficient) evidence that $\Delta G \neq 0$.
- (c) **Lyapunov argument.** If $\Delta G \equiv 0$, the map G would be constant, implying all trajectories converge to the same point. This contradicts the measured Lyapunov spectrum ($D_{KY} \approx 9$, positive exponents summing to ≈ 0.0037 nats/step), which implies local divergence of trajectories.

A fully rigorous proof of assumption (ii) via Gröbner basis computation or transfer-function analysis of the discrete SPDE as a pseudorandom permutation remains as a **reduced open problem**: the injectivity question has been reduced from “is G injective?” to “is ΔG a nonzero polynomial?”, a purely algebraic statement that does not require probabilistic arguments.

Consequence for Theorem 6.5

Under Proposition 9.9 and the extended Regime B argument, Theorem 6.5 is strengthened as follows.

Theorem 9.11 (Min-entropy of MFSU field, strengthened). *Assuming (A1) SHA3-512 is a random oracle, (A2) $\Delta G \neq 0$ (Proposition 9.9, assumption (ii)), and (A3) the Regime B rational extension is injective (Lyapunov argument above):*

$$H_\infty(F(k, IV) \mid k) \geq 512 \geq 128.$$

The bound holds analytically, not only empirically. The original Lemma 6.2 (empirical) is subsumed by this result and retained only as independent corroboration.

The proof reduces the injectivity question to assumption (ii), which is a concrete algebraic statement about a specific polynomial. This is strictly stronger than the empirical Lemma 6.2 but strictly weaker than a fully unconditional proof. A Gröbner basis computation for even a truncated version of G ($N = 4$, $M_0 = 2$ steps) would constitute formal verification of (ii). We consider this a tractable computational algebra problem and include it in the Year 1 roadmap.